

Team Mini-cappin

Marking Scheme

- 1.Repository Setup and CI: 2
- 2.User Stories Backlog: 3
- 3.Agile planning: 2
- 4.Software Architecture: 4
- 5.UI Prototypes: 5
- 6.Testing Report: 3
- 7.Issue tracking: 2
- 8.PR reviewing: 3
- 9.Code Quality report: 3
- 10.Presentation/Demo to the Product Owner/Teacher: 3

Comments

1.Repository Setup and CI [2/2]

The CI runs npx jest tests and upload the coverage report to codeCov
There are even actions for Maestro e2e tests, and SonarQube scanner.

2.User Stories Backlog [2/3]

I managed to find user stories with detailed acceptance criteria

<https://github.com/vibqetowi/Minicap/issues/25>

<https://github.com/vibqetowi/Minicap/issues/24>

<https://github.com/vibqetowi/Minicap/issues/23>

<https://github.com/vibqetowi/Minicap/issues/22>

<https://github.com/vibqetowi/Minicap/issues/20>

<https://github.com/vibqetowi/Minicap/issues/15>

All user stories have been created as issues, but not all of them have acceptance criteria.

Those that have are excellent and are further decomposed into sub-tasks.

3.Agile planning [1.5/2]

Please follow better the template I gave for Sprint tables.

Each Sprint Table should have only the user stories you planned for this sprint, their USPs and the status.

For Sprint 2, 3 out of 6 planned user stories were not completed. Some of their sub-tasks were not completed.

The retrospectives are well done. Please try to explain the issues in more detail. Most issues are explained in 2–3-word phrases.

It is good that you investigated GitHub projects for planning. I would like to have your feedback about it. Maybe we can adopt it for the next year.

4.Software Architecture [3.5/4]

You did exceptional work on the design diagrams.

Nice analysis of the MVVM architectural pattern and application to your project.

For the next release, the design patterns should include your own pattern implementations, and not those built-in the frameworks you use.

The technology stack link you provide is not exactly a discussion of the pros and cons of the libraries you depend on.

5.UI Prototypes [5/5]

Excellent work on UI prototyping. The analysis of personas is very good.
The mockups are documented as expected. You already have two version of mockups.

6. Testing Report [3/3]

20 unit tests with 88% coverage.

There are already 2 recorded end-to-end tests.

7. Issue tracking [2/2]

The is only one bug created, one day before the deadline. The bug is well documented. There is a template used.

<https://github.com/vibqetowi/Minicap/issues/114>

8. PR reviewing [2.5/3]

Most of the PRs have discussions. Most comments are about functionality issues, which makes sense. I could not find many comments discussing how to improve code quality.

<https://github.com/vibqetowi/Minicap/pull/113>

9. Code Quality report [3/3]

Excellent reporting and analysis. Please provide the link to SonarCloud reports.

10. Presentation/Demo to the Product Owner/Teacher [3/3]

	1	2	3	4	5	N/A
Time taken for the presentation (punctuality)					X	
Content coverage as requested (Did the team cover all expected topics?)					X	
Preparation for delivering the presentation (how well practiced)					X	
Coordination among presenters					X	
Presentation skills					X	
Answering of questions (Professionalism, clarity, sufficient explanation)					X	

Teacher comments:

The presentation has detailed information and good quality slides.

The presenters are well prepared.

The changes made to UI prototypes have been explained well.

There are 20 unit tests according to the presentation + End-to-end tests.

There is a detailed analysis of the issues reported by SonarQube.

The team demonstrated GitHub projects.

Finished on time 19 minutes.

Total: 27.5/30